TD 4 : Définition de données, Contraintes

Définition de données, Contraintes

2026-10-23

Avec solutions

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle

Prérequis

- schéma `world`,
- cours Création de tables, Insertion, Suppression, Mises à jour.
- cours Contraintes, Check, Primary Key, Foreign Key.

Contraintes SQL

Documentation PostgreSQL.

Contraintes CHECK.

Cette contrainte permet de vérifier que les colonnes d'une ligne donnée vérifient une certaine condition. Par exemple, on pourrait vouloir vérifier que la valeur de la colonne `percentage` de la table `countrylanguage` est inférieure à 100 ou que la valeur de la colonne `return_date` d'une table `rental` (pour stocker des locations) est `NULL` ou supérieure à `rental_date`. Ces contraintes se définissent lors de la création de la table. Par exemple :

```
CREATE TABLE solde(
    produit TEXT,
    reduc_pourcentage INT,
    CHECK (reduc_pourcentage<=100 AND reduc_pourcentage%10=0)
);
```

On peut ajouter une contrainte CHECK à une table qui existe déjà. Par exemple :

```
ALTER TABLE solde ADD
    CHECK (reduc_pourcentage >=0);
```

Il est préférable de vérifier qu'une table existe avant de tenter d'ajouter une contrainte.

```
ALTER TABLE IF EXISTS solde ADD
    CHECK (reduc_pourcentage >=0);
```

ou bien en nommant la contrainte :

```
ALTER TABLE IF EXISTS solde
    ADD CONSTRAINT reduc_positive
       CHECK (reduc_pourcentage >=0);
```

Contraintes de colonne `DEFAULT / NOT NULL / UNIQUE`

Ces contraintes s'appliquent à une colonne en particulier.

- Une contrainte `DEFAULT` spécifie la valeur par défaut d'une colonne lorsque sa valeur n'est pas spécifiée,
- Une contrainte `NOT NULL` spécifie que la valeur de la colonne ne peut pas être `NULL`
- Une contrainte `UNIQUE` spécifie que deux lignes différentes de la table ne peuvent pas avoir la même valeur pour les colonnes indiquées.

Par exemple :

```
CREATE TABLE membre(
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    date_inscription DATE DEFAULT NOW(),
    UNIQUE(nom, prenom)
);
```

Contraintes de table `PRIMARY/FOREIGN KEY`

- Formellement, la contrainte `PRIMARY KEY` est équivalente à `UNIQUE` et `NOT NULL`. Elle spécifie une ou plusieurs colonnes qui permettent d'identifier de façon unique chaque ligne. Dans une base de données bien conçue, chaque table doit avoir une clé primaire.
- On ne peut définir qu'une seule contrainte `PRIMARY KEY` sur une table.
- La contrainte `FOREIGN KEY` spécifie que la valeur d'une (ou de plusieurs) colonne contient des valeurs doivent être présentes dans une autre table. Par exemple, la colonne `capital` de `country` contient des valeurs qui doivent être présentes dans la colonne `id` de `city`.
- On peut définir plusieurs contraintes `FOREIGN KEY` sur une table.

Exemple :

```
CREATE TABLE IF NOT EXISTS membre(
    id INT,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
   PRIMARY KEY (id)
);

-- si la clé primaire ne contient qu'une colonne,
-- on peut aussi la définir au niveau de la colonne
-- idem pour la clé étrangère
CREATE TABLE IF NOT EXISTS amis(
    id1 INT PRIMARY KEY,
    id2 INT,
   FOREIGN KEY (id1) REFERENCES membre (id)
);
```

> **i** Ici, on a ajouté la clause `IF NOT EXISTS` pour éviter une erreur d'exécution si la table existe déjà. Dans ce cas, la requête ne fait rien. Si on veut recréer une table, il faut d'abord la supprimer.

On peut aussi ajouter ces contraintes après la définition de la table (il est alors préférable de vérifier que la table existe avant de tenter d'ajouter une contrainte) :

```
ALTER TABLE IF EXISTS amis ADD
    PRIMARY KEY (id1);
```

```
ALTER TABLE IF EXISTS amis ADD
    FOREIGN KEY (id2) REFERENCES membre;
```

Contraintes utilisant une fonction

Les contraintes `NOT NULL, DEFAULT, KEY, CHECK` portent uniquement sur une ligne tandis que la contrainte `UNIQUE` permet uniquement de tester des égalités entre lignes. On veut parfois vérifier une condition plus riche entre des lignes. Par exemple, si notre table contient des réservations d'hôtel entre `start_date` et `end_date`, on ne veut pas que deux réservations se chevauchent.

On peut alors utiliser une fonction dans une contrainte `CHECK` :

```
create table if not exists reservation(
   id int primary key,
   start_date date not null,
   end_date date not null check (end_date > start_date)
);

create or replace function compat (id_res int, start_res date, end_res date)
returns boolean
language sql as
$$
   select count (*) = 0
   from reservation r
   where id_res <> r.id
      and ((start_res >=  r.start_date
            and start_res <  r.end_date)
         or (r.start_date >=  start_res
            and r.start_date <  end_res)
         );

$$
-- il est possible aussi d'utiliser daterange(start_date, end_date)

alter table if exists reservation
   add constraint check_compat check (compat (id,start_date,end_date)) ;

insert into reservation values(1,'01/01/2026','01/10/2026'); -- est accepté
insert into reservation values(2,'01/10/2026','01/12/2026'); -- est accepté
insert into reservation values(3,'01/05/2026','01/10/2026'); -- est refusé
insert into reservation values(4,'12/30/2025','01/02/2025'); -- est refusé
```

> **Mode de vérification des contraintes `CHECK`**
>
> Les contraintes `CHECK` sont vérifiées uniquement lors de l'insertion et de la mise à jour. Les données de la table utilisées par la fonction sont celles existantes avant l'insertion et la mise à jour. **Les paramètres de la fonction sont les attributs de la ligne que l'on veut insérer ou mettre à jour**.

Contraintes `EXCLUDE`

La clause `EXCLUDE` permet de définir des contraintes d'exclusion entre deux lignes quelconques d'une table, c.a.d. que pour tout couple de lignes de la table, la propriété est exclue.

On obtient une contrainte équivalente à la précédente avec la définition suivante :

```
create table if not exists reservation(
   id int primary key,
   start_date date not null,
   end_date date not null check (end_date > start_date),
   exclude using gist (
```

```
        daterange(start_date, end_date) with &&)
);
```

Explication :

- `GiST` définit un type d'index pour effectuer les recherches. `GiST` comme "Generalized Search Tree" désigne une catégorie d'arbres (cf. Wikipedia).
- `daterange` calcule un intervalle de dates.
- `&&` est l'opérateur de comparaison utilisé. En PostgreSQL, il s'agit du test de recouvrement entre deux `daterange`.
- `exclude using gist (daterange(start_date, end_date) with &&)` signifie que pour deux lignes `r1` et `r2`, il est interdit que les intervalles `daterange(r1.start_date, r1.end_date)` et `daterange(r2.start_date, r2.end_date)` se recouvrent.

Voir la documentation PostgreSQL.

Exercices

On va travailler sur une copie du schéma `world`. Il va donc falloir copier dans vos schémas les tables `world.country`, `world.city` et `world.countrylanguage`.

```
CREATE TABLE IF NOT EXISTS td_country (like world.country) ;
```

La clause `like` permet de copier la structure d'une table existante dans une nouvelle table avec les types des colonnes et les contraintes `NOT NULL`. Mais aucune autre contrainte.

Voir la documentation de la clause `like`

Par contre, l'option `including all` permet de copier toutes les contraintes SAUF celles qui font référence à d'autres tables (clé étrangère, ...) :

```
CREATE TABLE IF NOT EXISTS td_country (
    like world.country
    including all
) ;
```

> **Vérifier la définition d'une table existante avec dBeaver**
>
> Avec dBeaver, vous pouvez visualiser la définition d'une table avec toutes ses contraintes en développant le schéma qui la contient puis "tables" puis la table qui vous intéresse.
> Ensuite :
> - ou bien dans la table, développer "DDL" et vérifier la définition de la table en SQL.
> - Ou bien dans la table, développer "Colonnes", "Contraintes" et "Clés étrangères" pour visualiser toutes les contraintes définies sur la table.

Questions

> Modifiez le `search_path` pour inclure votre schéma personnel.

1. Créer dans votre schéma trois tables `td_country`, `td_countrylanguage` et `td_city` de mêmes structures que celles de `world`.

> **Solution**
>
> ```
> CREATE TABLE IF NOT EXISTS td_country (like world.country including all);
> CREATE TABLE IF NOT EXISTS td_city (like world.city including all);
> CREATE TABLE IF NOT EXISTS td_countrylanguage
>                             (like world.countrylanguage including all);
> ```

2. Insérer dans les nouvelles tables les valeurs des tables originelles du schéma `world`.

Solution

```
INSERT INTO td_country
    SELECT * FROM world.country ;

INSERT INTO td_countrylanguage
    SELECT * FROM world.countrylanguage;

INSERT INTO td_city
    SELECT * FROM world.city;
```

3. Recréer les contraintes qui n'ont pas été copiées pour obtenir des tables qui soient réellement identiques. Vérifier cette identité en utilisant la note précisée en début d'exercice.

Solution

```
ALTER TABLE td_country
    ADD FOREIGN KEY (capital)
        REFERENCES td_city (id);

ALTER TABLE td_countrylanguage
    ADD FOREIGN KEY (countrycode)
        REFERENCES td_country (countrycode);
```

4. Ajouter la contrainte : la colonne `percentage` de `td_countrylanguage` contient un réel compris entre 0 et 100.

Solution

```
alter table td_countrylanguage
    add constraint td_countrylanguage_check_percentage
      check  (cast(0.0 as real)  <= percentage and
                percentage <= cast (100 as real)
) ;
```

L'ajout de cette contrainte échoue car certaines lignes ne la vérifie pas. Il faut supprimer ces lignes avant de pouvoir créer la contrainte !

5. Comment ajouter une contrainte `unique` sur le couple `(countrycode, language)` de la table `td_countrylanguage` ? Est-il utile d'ajouter cette contrainte à celles préalablement définies ?

Solution

```
alter table td_countrylanguage
    add unique (countrycode, language)  ;
```

L'ajout de cette contrainte est inutile car `(countrycode, language)` est déjà une `PRIMARY KEY`.

6. Mettre à jour dans `td_city` les populations des capitales du continent `Africa` en les multipliant par 1,1. Après cette mise à jour, la population de Djibouti est 421 300.

Solution

```
update td_city
set population = population * 1.1
from td_country
where td_country.capital = td_city.id
   and td_country.continent = 'Africa';
```

7. Supprimer les pays de la table `td_country` où aucune langue n'est parlée et qui n'ont pas de capitale (utiliser des CTEs). (6 lignes supprimées).

Solution

Avec 2 CTEs :

```
with country_without_language as
(
   select c.countrycode
   from td_country c left join td_countrylanguage cl
                     on c.countrycode = cl.countrycode
   where cl.countrycode is null
),
country_without_capital as
(
   select countrycode
   from td_country
   where capital is null
)
delete from td_country
using (country_without_language natural join country_without_capital) as s
where td_country.countrycode = s.countrycode;
```

Avec 1 CTE :

```
with a_supprimer as
(
   select c.countrycode
   from td_country c left join td_countrylanguage cl
                     on c.countrycode = cl.countrycode
   where cl.countrycode is null and c.capital is null
)
delete from td_country
using a_supprimer as s
where td_country.countrycode = s.countrycode;
```

8. Réécrire la requête d'insertion des données de la table `world.country` dans la table `td_country` en filtrant les lignes qui créeraient des doublons sur les colonnes de la table `td_country` ayant une contrainte `UNIQUE`. On suppose que les lignes insérées vérifient entre elles les contraintes d'unicité et on souhaite vérifier qu'elles ne créent pas de doublons avec les lignes présentes dans la table `td_country` avant l'insertion. Après la suppression de la question précédente, réinsérer avec cette nouvelle requête, les données de la table `world.country`. Si votre requête est correcte, son exécution ne doit pas provoquer de message d'erreur et à l'issue, les tables `world.country` et `td_country` doivent contenir les mêmes lignes.

La colonne `capital` a une contrainte `UNIQUE` mais pas de contrainte `NOT NULL`.

On peut tester l'identité des lignes des deux tables avec un `FULL JOIN` :

```
select *
from world.country c full join td_country tc using(countrycode)
where c.countrycode is null or tc.countrycode is null;
```

Le résultat doit être vide.

Solution

Une **erreur courante** est d'écrire :

```
insert into td_country
(
   select *
   from world.country as c
   where c.countrycode not in (select countrycode from td_country)
      and c.capital not in (select capital from td_country)
);
```

Si `c.capital` est `null` ou si le résultat de la requête `select capital from td_country` contient `null` la condition `not in` n'est pas vrai donc la ligne n'est pas sélectionnée.
Pour gérer les valeurs `null`, il faut écrire :

```
insert into td_country
(
   select *
   from world.country as c
   where c.countrycode not in (select countrycode from td_country)
      and c.capital is null
      or c.capital not in
         (select capital from td_country where capital is not null)
);
```

Autre méthode sans requête imbriquée :

```
with countrycode_absents as
(
   select c.countrycode
   from world.country as c
      left join td_country as tc
      on c.countrycode = tc.countrycode
   where tc.countrycode is null
),
capital_absentes as
(
   select c.countrycode
   from world.country as c
      left join td_country as tc
      on c.capital = tc.capital
   where tc.countrycode is null
)
insert into td_country
(
   select *
   from world.country as c
      natural join countrycode_absents
      natural join capital_absentes
);
```

9. Ajouter la contrainte : dans `td_country`, pour chaque pays, il n'existe pas un autre pays de la même `region` qui n'est pas dans le même `continent`. Il s'agit d'une contrainte correspondant à la *dépendance fonctionnelle* `region → continent`, c.a.d. que tous les pays d'une même région

doivent être sur un même continent.

Après la définition de la contrainte, testez l'insertion suivante :

```
insert into td_country(countrycode, name_country, continent, region, code2,
                localname, governmentform, population_country, surfacearea)
(
   select 'FR2', name_country, 'Asia', region, code2, localname,
            governmentform, population_country, surfacearea
   from td_country
   where countrycode = 'FRA'
);
```

Vous devez obtenir une message d'erreur indiquant que la contrainte définie n'est pas respectée.

Solution

```
create or replace function check_reg (reg text, conti text)
returns boolean
language sql as
$$
select
    count (*) = 0
from
    td_country a
where
    a.region = reg and
    not a.continent = conti;
$$

alter table if exists td_country
    add constraint RegionContinent check (check_reg (region, continent)) ;
```

OU

```
ALTER TABLE IF EXISTS td_country
    ADD CONSTRAINT df_region_continent
        EXCLUDE USING gist (
                region WITH =,
                continent WITH <>
) ;
```

La seconde formulation de la contrainte souligne qu'il s'agit d'une contrainte de table et pas d'une contrainte de ligne.